2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC) | 979-8-3315-6571-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/MCSOC67473.2025.00114

# FPGA Implementation of Tiny Transformer Using High-Level-Synthesis for Biomedical Applications

Mostafa Elsharkawy
*Nano-Electronics Engineering and Design*
*Singapore University of Technology and Design*
Singapore
*Chang Gung University*
Taiwan

Tee Hui Teo*
*Engineering Product Development*
*Singapapore University of Technology and Design*
Singapore
*Corresponding Author: tthui@sutd.edu.sg

I-Chyn Wei
*Department of Electrical Engineering*
*Chang Gung University*
Taiwan

***Abstract*—This paper presents the FPGA implementation of the Tiny Transformer for ECG arrhythmia detection on the Xilinx Kria KV260. The FPGA implementation using Vitis HLS resulted in 56.62 ms latency at 76.9 MHz. The design utilized balanced resources (30% LUTs, 37% BRAM, 27% DSP, and 20% Flip-Flops). Performance analysis reveals the feed-forward network dominated computational cost (54.9%), with inefficient memory access patterns requiring 36,261 iterations. Key bottlenecks include a lack of parallel attention head processing and insufficient exploitation of FPGA spatial parallelism. Despite current limitations, this work establishes a foundation for FPGA-based transformer inference in edge biomedical applications and identifies critical optimization opportunities for achieving competitive real-time performance.**

***Index Terms*—ECG analysis, edge computing, FPGA, high-level synthesis, transformer.**

## I. Introduction

Wearable systems are increasingly vital for continuous and real-time monitoring of cardiovascular diseases such as arrhythmia, providing valuable support for diagnosis and therapy. Electrocardiographic (ECG) signals are a widely accepted, non-invasive diagnostic tool, and their early and accurate analysis can help reduce the risks associated with heart anomalies, [1]. In this context, near-sensor edge processing has gained attention, as it enables continuous monitoring while preserving data privacy.

Machine learning models, and particularly transformers, have emerged as powerful tools for time-series classification [2]. Their attention mechanism enables the capture of non-local dependencies in sequential data, making them well-suited not only in natural language processing (NLP), vision processing [3], but also for biomedical image / signal processing [4]. Recent studies have shown that transformers can surpass Convolutional Neural Networks (CNNs) in arrhythmia detection, [5]. However, the computational and memory demands of conventional transformer architectures make them unsuitable for deployment on resource-constrained edge devices, where lightweight models remain the dominant choice despite their suboptimal accuracy, [6], [7].

To address this challenge, Busia et al. introduced the Tiny Transformer, a lightweight model specifically optimized for low-power edge devices [8]. Their design reduces model complexity to only ∼6k parameters while maintaining high accuracy (98.97% on the MIT-BIH arrhythmia dataset). Notably, the Tiny Transformer achieved real-time performance on the GAP9 microcontroller.Other works also show how transformer-based architectures can be adapted for microcontroller [9], [10].

While microcontrollers offer low energy consumption, Field-Programmable Gate Arrays (FPGAs) present a compelling alternative for edge AI workloads. Their inherent parallelism, reconfigurability, and higher computational throughput make them attractive for accelerating models that remain challenging on MCU platforms. In particular, AI-oriented SoC FPGAs such as the Xilinx Kria KV260 Starter Kit combine programmable logic with embedded processing capabilities, making them promising targets for edge deployment of complex neural networks [11].

The Tiny Transformer has been validated on microcontroller-based systems. This work aims to explore alternative hardware targets to further improve the response time and efficiency of the Tiny Transformer. The research question: Can the advantages of the Tiny Transformer be extended to FPGA platforms, and what challenges arise in doing so? Moving forward, the implementation of application-specific integrated circuits (ASICs) leveraging this lightweight Transformer will be carried out using an open platform [12], [13] to benefit the affordable healthcare community.

This work explores the deployment of the Tiny Transformer on the Xilinx Kria KV260 FPGA using Vitis High-Level Synthesis (HLS). The main objectives are:

1) **Implementation** – Translate the Tiny Transformer architecture into synthesizable hardware using Vitis HLS and targeted for the Kria KV260.

2771-3075/25/$31.00 ©2025 IEEE
DOI 10.1109/MCSoC67473.2025.00114

2) **Evaluation** – Measure estimated inference latency and resource utilization of the FPGA implementation.
3) **Analysis** – Identify the sources of performance bottlenecks, including the observed $\sim 12\times$ increase in latency compared to the MCU implementation.
4) **Future Directions** – Outline optimization strategies (e.g., pipelining, memory partitioning, quantization) to improve real-time feasibility.

Through this study, we provide an initial assessment of the opportunities and challenges of migrating highly efficient edge-oriented transformer models from microcontrollers to FPGA platforms, contributing insights for future FPGA-based accelerators for biomedical and edge AI applications.

The paper is organized as follows. In Section II, method, and implementation of the proposed design are discussed. The performance of the design is summarized in Section III. The paper is then concluded in Section IV.

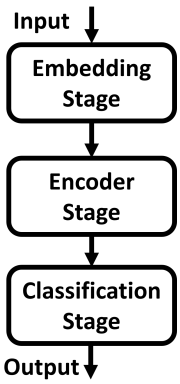


Fig. 1. Tiny Transformer workflow for ECG processing [8]

## II. Methods

The baseline model used in this work is the Tiny Transformer introduced by Busia et al. [8]. Unlike conventional transformer architectures that employ multiple encoder layers and large embedding dimensions, the Tiny Transformer was designed for execution on resource-constrained devices. Figure 1 presents the workflow the Tiny Transformer. It consists of:

- A 1D convolutional embedding stage that projects raw ECG input segments into patch embeddings.
- A single encoder block containing a Multi-Head Self-Attention (MHSA) mechanism and a lightweight feed-forward network.
- A classification head, implemented as a fully connected layer, for arrhythmia class prediction.

The model has approximately 6.6 k parameters, an embedding size of 16, and eight attention heads. This compact design makes it a promising candidate for FPGA acceleration, as it minimizes memory requirements while retaining the core attention mechanism.

### A. Tools and Hardware Platform

The FPGA implementation was carried out targeting the Xilinx Kria KV260 AI Starter Kit, a system-on-module platform based on the Zynq UltraScale+ MPSoC, which integrates programmable logic with ARM Cortex-A53 cores. The KV260 provides a flexible environment for deploying machine learning accelerators at the edge, [14].

High-Level Synthesis (HLS) was performed using Xilinx Vitis HLS 2023.1, which translates C/C++ functions into synthesizable HDL for FPGA implementation, [15]. This toolchain was selected to enable rapid prototyping and exploration of hardware design trade-offs without the need to code in Verilog or VHDL manually.

For functional verification, C/RTL co-simulation was employed within Vitis HLS. Synthesis and implementation reports were generated to extract latency, LUT/BRAM/DSP/FF utilization, and resource bottlenecks. Testing was performed with ECG input samples of fixed length, consistent with the original Tiny Transformer design.

### B. Implementation Flow

The model was partitioned into hardware modules corresponding to its main computational stages:

- **Embedding Layer**: Implemented as a 1D convolutional kernel using HLS #pragma HLS PIPELINE directives to maximize throughput. Loop unrolling was partially applied to balance parallelism against resource usage.
- **Multi-Head Self-Attention (MHSA)**: Each head was mapped as a separate computational unit, with shared memory for query, key, and value matrices. Matrix multiplications were accelerated using DSP slices, though bandwidth constraints limited achievable parallelism.
- **Feed-Forward Network**: Implemented as dense layers with ReLU activation. Loop tiling and pipelining were applied to improve performance.
- **Classification Layer**: A final fully connected layer mapped to a single HLS kernel, with negligible resource overhead compared to MHSA.

**Memory considerations**: On-chip BRAM was used to buffer intermediate tensors between layers to reduce off-chip DRAM accesses. However, due to the relatively high dimensionality of attention matrices, partial buffering was required, leading to increased data transfer latency.

**Optimization directives**: Initial attempts included loop pipelining, function inlining, and array partitioning to expose parallelism. Despite these, preliminary results showed that the FPGA implementation incurred approximately 12× higher latency than the MCU baseline, highlighting the need for further optimization, particularly in the MHSA block and memory hierarchy.

The FPGA implementation workflow is summarized in 2. The Tiny Transformer model was first translated into C/C++ functions, then synthesized into HDL using Vitis HLS. Functional verification and C/RTL co-simulation were performed and passed successfully, followed by synthesis reports for latency and resource analysis.

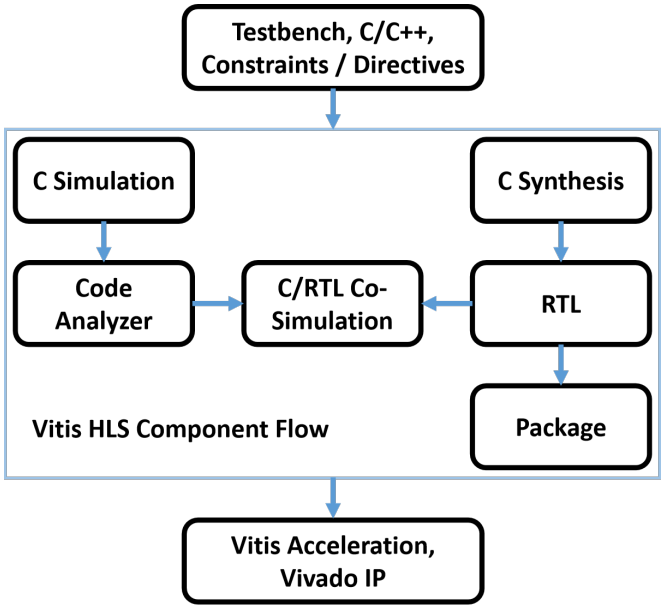


Fig. 2. Overview of the Tiny Transformer FPGA implementation workflow [15]

## III. Results and Discussion

The FPGA implementation of the tiny transformer achieved a total inference latency of 56.62 ms, operating at 76.92 MHz per inference. Table I presents a detailed breakdown of the computational latency distribution across the major components of the transformer.

The multi-head attention (MHA) module consumed 23.50 ms (41.5% of total inference time), while the feed-forward network required 31.11 ms (54.9%), making it the primary computational bottleneck. Layer normalization operations contributed minimally to the overall latency at 0.43 ms (0.8%). This distribution contrasts sharply with the expected behavior where attention mechanisms typically dominate transformer computational costs.

The synthesized design utilized moderate FPGA resources as detailed in Table II. The implementation consumed 35,881 LUTs (30% utilization), 109 BRAM blocks (37% utilization), 342 DSP slices (27% utilization), and 47,415 flip-flops (20% utilization). The balanced resource consumption across different FPGA primitives suggests that the design is not critically constrained by any single resource type, indicating potential for either design scaling or further optimization.

TABLE I
Latency Breakdown of Transformer Components

| Component | Latency (ms) | Cycles | Percentage (%) |
|---|---|---|---|
| Multi-Head Attention | 23.501 | 1 807 614 | 41.5 |
| Feed-Forward Network | 31.112 | 2 393 226 | 54.9 |
| Layer Normalization | 0.433 | 3397 | 0.8 |
| Input/Output Processing | 1.581 | 21 054 | 2.8 |
| Total | 56.624 | 4 355 291 | 100 |

TABLE II
FPGA Resource Utilization

| Resource Type | Used | Available | Utilization (%) |
|---|---|---|---|
| LUTs | 35 881 | 117.120 | 30 |
| BRAM | 109 | 288 | 37 |
| DSP | 342 | 1200 | 27 |
| Flip-Flops | 47 415 | 234.240 | 20 |

The performance analysis reveals several fundamental limitations in the current FPGA implementation. The synthesis results indicate predominantly sequential execution patterns, with only two operations exhibiting pipelining: the `VALUE_APPLICATION` loop (266 cycles) and `VITIS_LOOP_102_1` (11 cycles). This sequential processing approach fails to exploit the spatial parallelism capabilities inherent in FPGA architectures.

Memory access patterns emerge as a significant bottleneck, evidenced by high iteration counts in critical computational loops. The `QKV_COMPUTATION` requires 6,307 iterations per 66-patch sequence, while `HEAD_PROCESSING` demands 2,362 iterations for attention computation. The feed-forward network exhibits the most concerning behavior with 36,261 iterations, suggesting inefficient memory access patterns that likely contribute to its unexpected dominance in the latency profile.

The attention mechanism implementation particularly underperforms expectations. Rather than processing multiple attention heads in parallel—a key architectural advantage of transformers—the current implementation processes heads sequentially. This design choice negates one of the primary benefits of the transformer architecture and represents a missed optimization opportunity.

The performance of the Tiny Transformer in FPGA platform can be improved by substantial architectural design as enhanced pipelining strategies, parallel attention head processing, and optimized memory access patterns tailored to transformer computational characteristics.

## IV. Conclusion

This work presents the first FPGA implementation of the Tiny Transformer on the Xilinx Kria KV260 using Vitis HLS. The functionality of the design was verified and comparable to the GAP9 microcontroller implementation. Future work is to improve the inference speed by exploiting FPGA parallelism. The resource utilization is balanced (30% LUTs, 37% BRAM, 27% DSP), indicating that performance is constrained by architectural choices and synthesis inefficiencies, particularly in the feed-forward network. Despite these limitations, this study establishes a framework for FPGA-based transformer inference and highlights key bottlenecks and optimization challenges for edge AI acceleration.

## References

[1] T. H. Teo, X. Qian, P. Kumar Gopalakrishnan, Y. S. Hwan, K. Haridas, C. Y. Pang, H.-K. Cha, and M. Je, "A 700-uW Wireless Sensor Node SoC for Continuous Real-Time Health Monitoring," *IEEE*

*Journal of Solid-State Circuits*, p. 5607234, 2010. [Online]. Available: http://ieeexplore.ieee.org/document/5607234/

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. (2017) Attention Is All You Need. [Online]. Available: https://arxiv.org/abs/1706.03762

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. (2020) An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. [Online]. Available: https://arxiv.org/abs/2010.11929

[4] Q. Pu, Z. Xi, S. Yin, Z. Zhao, and L. Zhao, "Advantages of transformer and its application for medical image segmentation: A survey," *BioMedical Engineering OnLine*, vol. 23, no. 1, p. 14, 2024. [Online]. Available: https://biomedical-engineering-online.biomedcentral.com/articles/10.1186/s12938-024-01212-4

[5] J. Zhang, A. Liu, M. Gao, X. Chen, X. Zhang, and X. Chen, "ECG-based multi-class arrhythmia detection using spatio-temporal attention-based convolutional recurrent neural network," *Artificial Intelligence in Medicine*, vol. 106, p. 101856, 2020. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0933365719312606

[6] T. H. Teo, W. M. Tan, and Y. S. Tan, "Tumour Detection using Convolutional Neural Network on a Lightweight Multi-Core Device," in *2019 IEEE 13th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*. Singapore, Singapore: IEEE, Oct. 2019, pp. 87–92.

[7] X. Wang, Y. Zhu, Y. Cui, X. Huang, D. Guo, P. Mu, M. Xia, C. Bai, Z. Teng, and S. Chen, "Lightweight Multi-Stage Aggregation Transformer for robust medical image segmentation," *Medical Image Analysis*, vol. 103, p. 103569, 2025. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1361841525001161

[8] P. Busia, M. A. Scrugli, V. J.-B. Jung, L. Benini, and P. Meloni, "A Tiny Transformer for Low-Power Arrhythmia Classification on Microcontrollers," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 19, no. 1, pp. 142–152, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10531812/

[9] J. Yang, J. Liao, F. Lei, M. Liu, J. Chen, L. Long, H. Wan, B. Yu, and W. Zhao. (2023) TinyFormer: Efficient Transformer Design and Deployment on Tiny Devices. [Online]. Available: https://arxiv.org/abs/2311.01759

[10] V. J.-B. Jung, A. Burrello, M. Scherer, F. Conti, and L. Benini, "Optimizing the Deployment of Tiny Transformers on Low-Power MCUs," *IEEE Transactions on Computers*, vol. 74, no. 2, pp. 526–541, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10755971/

[11] R. Li and S. Chen. (2025) Design and Implementation of an FPGA-Based Hardware Accelerator for Transformer. [Online]. Available: https://arxiv.org/abs/2503.16731

[12] T. H. Teo, M. Xiang, E. Goh, and H.-K. Hsu, "Open-AI Driven Open-source Open-access Sustainable ICs Design Flow," in *2024 IEEE 17th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*. Kuala Lumpur, Malaysia: IEEE, Dec. 2024, pp. 361–365.

[13] T. H. Teo, K. Qian, K. Li, and C.-Y. Li, "Open-Source Process Design Kits in Power Management Circuits for Educational Purposes," in *2025 IEEE 15th International Conference on Power Electronics and Drive Systems (PEDS)*. Penang, Malaysia: IEEE, Jul. 2025, pp. 1–5.

[14] Kria KV260 Vision AI Starter Kit. AMD. [Online]. Available: https://www.amd.com/en/products/system-on-modules/kria/k26/kv260-vision-starter-kit.html

[15] Introduction Vitis High-Level Synthesis User Guide (UG1399) Reader AMD Technical Information Portal. [Online]. Available: https://docs.amd.com/r/en-US/ug1399-vitis-hls